

2026 년 전기 졸업과제 착수보고서

멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 과정 개선 지원 기술 연구



팀명 : EasyDep

202113111 김재우 (projw1024@pusan.ac.kr)

202155578 이상현 (ask0208ask@pusan.ac.kr)

202055625 황수환 (tnghks2866@pusan.ac.kr)

목차

1. 과제의 목표.....	3
1) 과제 배경.....	3
2) 과제 세부 목표.....	3
2. 대상 문제 및 요구사항 분석.....	4
1) 유사 시스템 분석.....	4
2) 문제점 분석.....	5
3) 시스템의 필요성.....	5
4) 요구사항 분석.....	6
5) 유스케이스 분석.....	8
3. 현실적 제약 사항 분석 결과 및 대안.....	20
1) 현실적 제약 사항.....	20
2) 대안.....	20
4. 시스템 구성.....	21
1) 시스템 구성도.....	21
2) 개발 환경.....	22
3) 사용 기술.....	24
5. 개발 일정 및 역할 분담.....	30
1) 개발 일정.....	30
2) 역할 분담.....	31

1. 과제의 목표

1) 과제 배경

클라우드 네이티브 애플리케이션은 클라우드 환경의 이점(가용성, 서비스 간 연결성 등)을 활용할 수 있도록 개발하는 소프트웨어 산출물을 말한다. 클라우드 네이티브 애플리케이션 구축 시, 기반이 되는 클라우드 리소스(네트워크, 가상머신 등) 적합성과 함께 사용자 요구사항을 만족시키는 소프트웨어 개발 과정이 도입될 필요가 있다. 이를 수행하기 위한 방안으로 대규모 언어 모델(LLM, Large Language Model)의 자연어 해석 능력을 통해 분야별 특성을 추가적으로 학습한 AI 에이전트 기술을 활용할 수 있다. AI 에이전트는 초기 산출물을 짧은 시간 내에 도출할 수 있으며, 이는 소프트웨어 개발 프로세스의 전체적인 주기를 감소시켜 소프트웨어 품질 개선 측면에 도움을 줄 수 있다.

그러나 현재 개발되는 AI 에이전트 기반 소프트웨어 개발 지원 기술들은 단일 서비스 개발 지원에 초점을 두고 있다. 즉, 클라우드 네이티브 애플리케이션과 같이 다양한 유형의 클라우드 인프라, 리소스, 서비스 등의 종합적인 고려가 필요한 경우에는 정확한 소프트웨어 산출물을 기대하기 힘들다. 또한, LLM의 특성 중 하나인 환각(Hallucination) 현상에 따라, 사용자 요구사항에 부합하지 않는 결과를 도출할 수 있다. 즉, 사용자가 활용하고자 하는 클라우드 리소스 및 서비스와 달라질 수 있으며, 이는 클라우드 환경에 적합한 서비스를 구축해야 하는 클라우드 네이티브 애플리케이션에서 결함이 발생할 수 있다. 마지막으로, 클라우드 네이티브 애플리케이션 개발 시 고려해야 하는 다양한 형태의 산출물(다이어그램, API 명세, 인프라 구성 파일 등)은 AI 에이전트 구성을 더욱 어렵게 만드는 원인이다. 따라서, 클라우드 네이티브 애플리케이션을 지원하는 AI 에이전트는 개발에 필요한 산출물들을 구성 요소 및 개발 과정에 따라 체계적으로 검증할 수 있는 기술이 요구된다.

따라서, 본 과제에서는 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 과정 개선 지원 기술을 제안한다. 본 과제는 1) 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 단계별 요구사항 수립 체계 구축, 2) 멀티 AI 에이전트 기반 소프트웨어 산출물 테스트 기법 도출, 3) 테스트 결과를 반영한 소프트웨어 개선 방법 개발을 목표로 수행한다.

2) 과제 세부 목표

- ① 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 단계별 요구사항 수립 체계 구축
- ② 멀티 AI 에이전트 기반 소프트웨어 산출물 테스트 기법 도출
- ③ 테스트 결과를 반영한 소프트웨어 개선 방법 개발

2. 대상 문제 및 요구사항 분석

1) 유사 시스템 분석

① MAFT(Multi Agents For Testing) : MAFT 는 LLM 기반 에이전트들이 E2E(End-to-End) 테스트 코드 생성을 여러 작은 작업들로 분리하고 수행하는 파이프라인이다.

i) 사용자로부터 API 문서 및 테스트 코드 생성 지침을 받아 E2E 테스트 코드를 생성한다.

ii) 테스트 코드에 상태를 원상복구하는 로직을 넣음으로써 각 테스트 케이스가 다른 테스트 케이스에 영향을 미치지 않도록 설계되었다.

② Testsigma: 자연어 기반의 통합 테스트 자동화 플랫폼이다.

i) 자연어:요구사항·디자인·사용 흐름 자료를 입력으로 받아 웹, 모바일, 데스크톱, API 테스트 케이스를 생성한다.

ii) UI 변경 등이 발생하면 그에 맞게 테스트를 스스로 수정하는 기능을 제공한다.

③ TestMu AI: LLM 기반 입력 해석과 테스트 실행 인프라를 결합하여 테스트 케이스 생성부터 자동화 및 실행을 지원하는 테스트 플랫폼이다.

i) 사용자 입력을 해석하여 구조화된 컨텍스트(Context)와 테스트 케이스를 생성한다.

ii) LLM 으로 생성된 테스트 케이스와 자연어로 된 테스트 시나리오를 사용자가 자연어로 수정 및 재구성할 수 있다.

2) 문제점 분석

- ① 클라우드 네이티브 애플리케이션을 위한 개발 지원 체계 부재: 기존 테스트 지원 플랫폼들은 단일 서비스 개발에 초점을 두고 있으며, 클라우드 네이티브 환경과 같은 분산된 서비스나 다양한 클라우드 컴퓨팅 리소스를 반영하여 애플리케이션 개발을 지원하기 위한 방법이 부족하다.
- ② AI 에이전트 생성 결과의 요구사항 만족도 검증 방법 부재: 소프트웨어 개발 과정은 단계별(요구사항 분석, 시스템 설계, 구현, 유지보수 등)로 다른 유형의 산출물이 도출되며, 각 산출물이 다음 단계에 영향을 미치게 된다. 이때 최초로 도출되는 결과물에서 오류가 발생하면 이후 산출물도 오류를 포함할 수 있다.
- ③ 다양한 형태의 개발 결과물: 최종 결과물인 클라우드 네이티브 애플리케이션은 다양한 형태의 산출물(요구사항 명세서, 시스템 설계를 위한 다이어그램, 소스 코드, 인프라 구성 파일 등)을 종합하여 구성된다. 즉, AI 에이전트가 최종 결과물에 맞춰 다양한 형태의 산출물을 해석하는 능력이 부족하다.

3) 시스템의 필요성

- ① 클라우드 네이티브 애플리케이션 특화 개발 지원 체계 필요: 클라우드 네이티브 애플리케이션은 다양한 클라우드 리소스와 인프라를 기반으로 구성된 분산 서비스이다. 클라우드 네이티브 애플리케이션의 구조적 특성(확장성, 탄력성, 장애 복원성, 기능 간 약결합 등)을 반영하고 개발 주기를 감소시킬 수 있는 AI 에이전트 기반의 개발 지원 체계가 필요하다.
- ② 소프트웨어 개발 과정별 요구사항 만족도 검증 체계 구축 필요: AI 에이전트 기반으로 클라우드 네이티브 애플리케이션을 개발하는 경우, 개발 단계별 산출물이 자동적으로 생성된다. 생성된 산출물들은 LLM의 특징인 환각 및 오류 전파 가능성이 존재하므로 요구사항에 맞추어 생성되었는지를 검증할 수 있는 체계가 필요하다.
- ③ 멀티 AI 에이전트 기반 소프트웨어 개발 산출물 피드백 구조 필요: 클라우드 네이티브 애플리케이션 개발 과정에서는 다양한 유형의 산출물들(코드, 아키텍처 모델, API 명세, 인프라 설정 파일 등)이 생성된다. AI 에이전트를 통해 고품질의 클라우드 네이티브 애플리케이션을 생성하기 위해서는 다양한 유형의 산출물들에 특화된 AI 에이전트를 활용하여 분석·검증·개선 작업을 수행할 수 있는 피드백 구조가 필요하다.

4) 요구사항 분석

① 기능적 요구사항

표 1 은 시스템이 제공해야 할 기능들에 대한 요구사항을 나타낸다.

표 1. 기능적 요구사항

기능		설명
요구사항 수립	클라우드 네이티브 애플리케이션 요구사항 입력	시스템은 개발하고자 하는 클라우드 네이티브 애플리케이션의 요구사항을 (ReqID: 요구사항)의 형태로 사용자에게 입력받아 DBMS 에 저장한다.
	요구사항 분석 산출물 도출	요구사항 분석 에이전트는 사용자의 입력을 바탕으로 요구사항 명세서를 생성하고 DBMS 에 저장한다.
산출물 관리	시스템 설계 산출물 도출	시스템 설계 에이전트는 요구사항 명세서를 바탕으로 시스템 아키텍처 다이어그램을 생성하고 DBMS 에 저장한다.
	서비스 개발 산출물 도출	서비스 개발 에이전트는 시스템 아키텍처 다이어그램과 요구사항을 기반으로 소스 코드 및 배포 파일을 생성하고 DBMS 에 저장한다.
	테스팅 산출물 도출	테스팅 에이전트는 생성된 산출물을 바탕으로 테스트가 수행되어야 할 범위와 기준 데이터를 산출하여 DBMS 에 저장한다.
	산출물 재생성	시스템은 테스트 결과 중 실패 항목이 존재할 경우, 관련 에이전트를 통해 산출물을 수정하여 DBMS 에 저장한다.
테스트 케이스 관리	테스트 케이스 생성	테스트 케이스 생성 에이전트는 각 단계별 산출물이 생성될 때마다 이를 기반으로 요구사항을 검증할 테스트 케이스를 생성하여 DBMS 에 저장한다.
	테스트 케이스 검증	시스템은 테스트 결과에 따라 산출물을 재생성(테스팅 결과 실패가 하나라도 포함)하거나 다음 개발 단계로 진행(모든 요구사항 부합)할 수 있어야 한다.
	테스트 수행	시스템은 생성된 테스트 케이스들을 기반으로 테스트를 수행하고 테스트 성공 여부를 반환할 수 있어야 한다.
애플리케이션 제공	클라우드 네이티브 애플리케이션 제공	시스템은 최종적으로 개발된 클라우드 네이티브 애플리케이션의 산출물들을 사용자에게 파일 형태로 제공할 수 있어야 한다.

② 비기능적 요구사항

표 2 는 시스템이 만족해야 할 비기능적 요구사항을 나타낸다.

표 2. 비기능적 요구사항

요건	설명
가용성	시스템은 사용자가 필요로 할 때 언제든지 접근 및 활용이 가능해야 하며, 99.9% 이상의 가용성을 보장해야 함
신뢰성	시스템은 AI 에이전트가 생성한 산출물의 결함을 방지하기 위해 사용자에게 산출물을 확인할 수 있는 과정이 포함되어야 함
사용 편의성	시스템은 개발 지식이 부족한 기획자나 관리자도 대화형 인터페이스를 통해 시스템 요구사항을 쉽게 구체화할 수 있도록 UI/UX 를 제공해야 함
성능	시스템은 사용자가 클라우드 네이티브 애플리케이션의 개발 산출물의 제공을 요청 할 때, 평균 5 초 이내에 제공할 수 있어야 함
유지보수성	시스템은 장애가 발생하더라도 최대 6 분 이내에 시스템을 정상화하여 복구 가능해야 함

5) 유스케이스 분석

① 유스케이스 다이어그램

그림 1 은 시스템의 유스케이스 다이어그램이다.

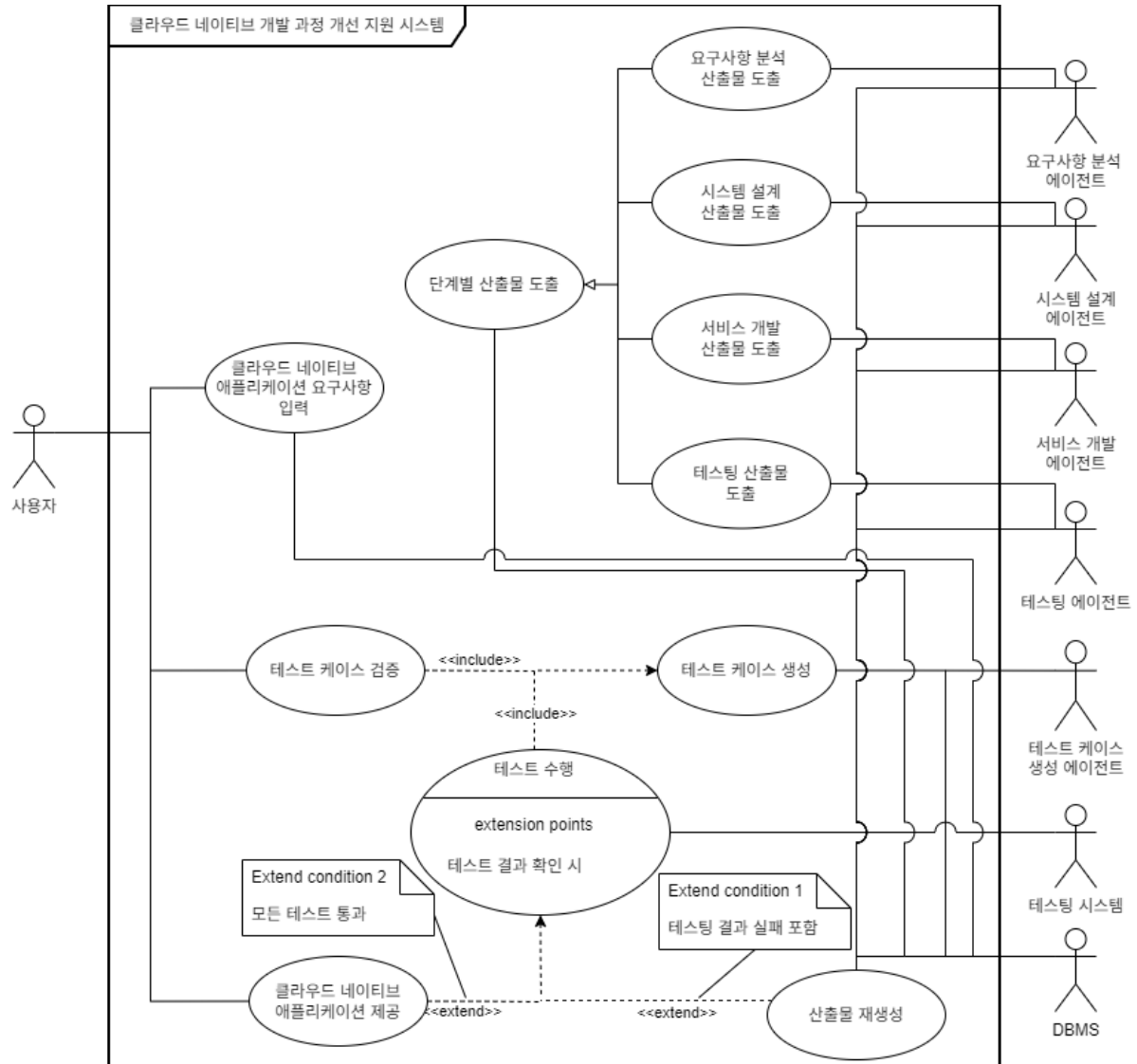


그림 1. 클라우드 네이티브 개발 과정 개선 지원 시스템의 유스케이스 다이어그램

② 액터(Actor)

표 3 은 시스템의 액터에 대한 설명을 나타낸 표이다.

표 3. 액터 명세

액터	설명
사용자	시스템에 애플리케이션 요구사항을 자연어로 입력하고, 생성된 테스트 케이스의 적합성을 검증하며, 최종 산출물 파일을 제공받는 시스템 이용 주체
요구사항 분석 에이전트	사용자의 클라우드 네이티브 애플리케이션 요구사항을 바탕으로 요구사항 분석 산출물을 생성하는 에이전트
시스템 설계 에이전트	사용자의 자연어 요구사항을 바탕으로 시스템 설계 산출물을 생성하는 에이전트
서비스 개발 에이전트	사용자의 자연어 요구사항을 바탕으로 서비스 개발 산출물을 생성하는 에이전트
테스팅 에이전트	사용자의 자연어 요구사항을 바탕으로 테스팅 산출물을 생성하는 에이전트
테스트 케이스 생성 에이전트	각 단계의 산출물을 바탕으로 해당 단계에 알맞은 테스트 케이스를 생성하는 에이전트
테스팅 시스템	산출물 에이전트들이 도출한 테스트 케이스를 바탕으로 산출물의 품질을 측정하는 시스템
DBMS	개발 전 주기에 걸쳐 생성되는 다양한 산출물, 테스트 케이스들을 저장하고 관리하는 외부 데이터베이스 시스템

③ 유스케이스 명세

● 클라우드 네이티브 애플리케이션 요구사항 입력

유스케이스명	클라우드 네이티브 애플리케이션 요구사항 입력
개요	시스템은 개발하고자 하는 클라우드 네이티브 애플리케이션의 요구사항을 사용자에게 입력받아 DBMS 에 저장한다.
관련 액터	사용자, DBMS
선행 조건	없음
이벤트 흐름	기본흐름: 1. 사용자는 시스템에 클라우드 네이티브 애플리케이션 개발 과정에 대한 요구사항을 (ReqID: 요구사항) 형태로 입력한다. 2. 시스템은 입력받은 요구사항을 저장하기 위해 unique 한 App ID 를 생성한다. 3. 시스템은 (App ID: [(ReqID: 요구사항), ...]) 형태로 DBMS 에 저장을 요청한다. 4. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 요구사항 입력 완료 메시지를 반환한다. 예외흐름: 4-1. 시스템이 DBMS 로부터 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	사용자가 입력한 요구사항이 식별자(App ID)와 함께 DBMS 에 저장된다.

● 요구사항 분석 산출물 도출

유스케이스명	요구사항 분석 산출물 도출
개요	요구사항 분석 에이전트가 DBMS 에 저장된 사용자의 요구사항을 분석하여 요구사항 명세서를 도출한다.
관련 액터	요구사항 분석 에이전트, DBMS
선행 조건	사용자의 요구사항 입력이 완료되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 DBMS 로부터 사용자가 요청한 App ID 의 요구사항 리스트의 조회를 요청한다. 2. 시스템은 DBMS 로부터 조회한 요구사항 리스트를 요구사항 분석 에이전트로 전달한다. 3. 시스템은 요구사항 분석 에이전트의 수행 결과로 (목적, 배경, 기능, 비기능)으로 구성된 요구사항 명세서(Spec Object)를 전달받는다. 4. 시스템은 (App ID: Spec Object) 형태로 DBMS 에 저장을 요청한다. 5. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 요구사항 분석 산출물 도출이 완료됨을 반환한다. 예외흐름: 3-1. 요구사항 분석 에이전트가 명세서 생성 중 분석 오류를 발생시킨 경우: 1. 시스템은 요구사항 분석 에이전트에게 재생성을 요청하며 재요청 횟수를 기록한다. 재요청이 3 회 이상일 경우 작업을 중단하고 사용자에게 요구사항 수정을 요청한다. 5-1. 시스템이 DBMS 로부터 데이터 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	구조화된 요구사항 명세서가 DBMS 에 저장된다.

● 시스템 설계 산출물 도출

유스케이스명	시스템 설계 산출물 도출
개요	시스템 설계 에이전트는 요구사항 명세서를 바탕으로 시스템 아키텍처 다이어그램을 생성하고 DBMS 에 저장한다.
관련 액터	시스템 설계 에이전트, DBMS
선행 조건	특정 App ID 의 요구사항 명세서가 DBMS 에 저장되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 DBMS 로부터 요구사항 명세서를 조회하여 시스템 설계 에이전트에게 전달한다. 2. 시스템 설계 에이전트는 전달받은 요구사항 명세서를 토대로 클라우드 환경에 적합한 배포 다이어그램, 클래스 다이어그램, 시퀀스 다이어그램, 유스케이스 다이어그램 및 API 명세를 생성한다. 3. 시스템은 (App ID: Diagram Object) 형태로 DBMS 에 저장을 요청한다. 4. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 시스템 설계 산출물 도출이 완료됨을 반환한다. 예외흐름: 4-1. 시스템이 DBMS 로부터 데이터 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	애플리케이션의 시스템 아키텍처 다이어그램 데이터가 DBMS 에 저장된다.

● 서비스 개발 산출물 도출

유스케이스명	서비스 개발 산출물 도출
개요	서비스 개발 에이전트는 시스템 아키텍처 다이어그램과 요구사항을 기반으로 소스 코드 및 배포 파일을 생성하고 DBMS 에 저장한다.
관련 액터	서비스 개발 에이전트, DBMS
선행 조건	특정 App ID 의 시스템 아키텍처 다이어그램이 DBMS 에 저장되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 DBMS 로부터 시스템 아키텍처 다이어그램 및 요구사항 데이터를 조회하여 서비스 개발 에이전트에게 전달한다. 2. 서비스 개발 에이전트는 사용자의 요구사항과 시스템 아키텍처 다이어그램을 분석하여 비즈니스 로직이 포함된 소스 코드와 배포 설정 파일을 생성한다. 3. 시스템은 생성된 파일 데이터를 (App ID: 소스 코드, 배포 설정 파일) 형태로 DBMS 에 저장을 요청한다. 4. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 서비스 개발 산출물 도출이 완료됨을 반환한다. 예외흐름: 4-1. 시스템이 DBMS 로부터 데이터 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	소스 코드 및 배포 설정 파일이 DBMS 에 저장된다.

● 테스트 산출물 도출

유스케이스명	테스팅 산출물 도출
개요	테스팅 에이전트는 생성된 산출물을 바탕으로 테스트가 수행되어야 할 범위와 기준 데이터를 산출하여 DBMS 에 저장한다.
관련 액터	테스팅 에이전트, DBMS
선행 조건	개발 단계별 산출물이 DBMS 에 저장되어 있어야 한다.
이벤트 흐름	기본흐름: 1. 테스트 에이전트는 DBMS 로부터 앞선 단계에서 생성된 요구사항 명세서와 개발 산출물(소스 코드, 아키텍처 설계서 등)을 로드한다. 2. 테스트 에이전트는 로드된 소스 코드의 논리 구조와 요구사항 명세서를 상호 매핑하여, 누락 없이 검증이 필요한 테스트 대상 및 범위를 식별한다. 3. 테스트 에이전트는 식별된 테스트 범위를 바탕으로 전체 시스템 대비 달성해야 할 테스트 커버리지 목표를 도출한다. 4. 테스트 에이전트는 도출된 테스트 산출물인 테스트 범위와 커버리지 목표를 관리하기 위해 (App ID: 테스트 산출물) 형태로 DBMS 에 저장을 요청한다. 5. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 테스트 산출물 도출 작업이 완료되었음을 사용자에게 알리고 다음 단계로 전환한다. 예외흐름: 5-1. 시스템이 DBMS 에 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	단계별 테스트 커버리지 목표 지표가 DBMS 에 저장된다.

● 산출물 재생성

유스케이스명	산출물 재생성
개요	시스템은 테스트 결과 중 실패 항목이 존재할 경우, 관련 에이전트를 통해 산출물을 수정하여 DBMS 에 저장한다.
관련 액터	요구사항 분석 에이전트, 시스템 설계 에이전트, 서비스 개발 에이전트, 테스트 에이전트, DBMS
선행 조건	테스트 수행 결과에 '실패' 항목이 하나 이상 포함되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 DBMS 로부터 테스트 결과 정보를 조회하여 실패 결과를 가진 ReqID 와 해당 에러 로그를 식별한다. 2. 시스템은 에러 로그와 관련 산출물을 해당 에이전트에게 전달하여 산출물 재생성을 요청한다. 3. 특정 에이전트는 결함이 수정된 새로운 산출물을 반환한다. 4. 시스템은 새로운 산출물 데이터를 DBMS 에 저장을 요청한다. 5. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 산출물 재생성이 완료됨을 반환한다. 예외흐름: 2-1. 시스템이 조회한 테스트 결과 정보에 해당하는 에이전트가 동일한 에러에 대해 산출물 재생성을 3 회 초과한 경우: 1. 시스템은 산출물 재생성 작업을 중단한다. 5-1. 시스템이 DBMS 에 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	결함이 수정된 새로운 버전의 산출물이 DBMS 에 저장되며, 테스트 수행 단계로 되돌아간다.

● 테스트 케이스 생성

유스케이스명	테스트 케이스 생성
개요	테스트 케이스 생성 에이전트는 각 단계별 산출물이 생성될 때마다 이를 기반으로 요구사항을 검증할 테스트 케이스를 생성하여 DBMS 에 저장한다.
관련 액터	테스트 케이스 생성 에이전트, DBMS
선행 조건	특정 개발 단계의 산출물 도출이 완료되어 DBMS 에 저장되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 새로운 산출물 도출을 감지하고 DBMS 로부터 산출물과 매핑된 원본 ReqID 데이터를 조회하여 테스트 케이스 생성 에이전트에게 전달한다. 2. 테스트 케이스 생성 에이전트는 각 ReqID 를 검증할 수 있는 테스트 케이스를 생성한다. 3. 시스템은 (App ID: [(ReqID: 테스트 케이스), ...]) 형태로 DBMS 에 저장을 요청한다. 4. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 테스트 케이스 생성이 완료됨을 반환한다. 예외흐름: 3-1. 시스템이 DBMS 로부터 데이터 저장 실패 메시지를 반환받은 경우: 1. 시스템은 데이터 저장을 재요청한다. (최대 3 회) 2. 재요청 중 저장이 성공하면 정상 흐름으로 복귀한다. 3. 3 회 재요청 후에도 저장이 실패할 경우, 시스템은 에러 로그를 기록한다. 4. 시스템은 진행 중인 작업을 중단한다.
후행 조건	요구사항 식별자와 매핑된 테스트 케이스가 DBMS 에 저장된다.

● 테스트 케이스 검증

유스케이스명	테스트 케이스 검증
개요	사용자는 생성된 테스트 케이스가 요구사항을 명확히 검증하는지 조회하고 승인 여부를 시스템에 전달한다.
관련 액터	사용자, DBMS
선행 조건	테스트 케이스가 생성되어 DBMS 에 저장되어 있어야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 시스템에 특정 App ID 의 테스트 케이스 목록 조회를 요청한다. 2. 시스템은 DBMS 로부터 데이터를 불러와 사용자 인터페이스에 출력한다. 3. 사용자는 내용 검토 후 '승인'을 선택하여 시스템에 전송한다. 4. 시스템은 DBMS 에 해당 테스트 케이스의 상태를 승인된 상태로 업데이트하도록 요청하고 성공 메시지를 반환받으면, 사용자에게 테스트 케이스 검증이 완료됨을 반환한다. 대안흐름: 3-1. 사용자가 테스트 케이스 내용에 오류가 있다고 판단하여 '반려'를 선택한 경우: 1. 시스템은 DBMS 에 테스트 케이스 상태를 반려 상태로 기록하고, 반려 사유를 테스트 케이스 생성 에이전트에게 전달하여 '테스트 케이스 생성' 유스케이스를 재실행한다.
후행 조건	테스트 케이스의 상태가 승인(또는 반려)으로 변경되어 기록된다.

● 테스트 수행

유스케이스명	테스팅 수행
개요	테스팅 시스템은 승인된 테스트 케이스를 바탕으로 테스팅을 수행하고 성공/실패 여부를 반환하여 DBMS 에 저장한다.
관련 액터	테스팅 시스템, DBMS
선행 조건	테스트 케이스가 사용자로부터 승인된 상태여야 한다.
이벤트 흐름	기본흐름: 1. 테스트 시스템은 DBMS 로부터 실행 환경에 필요한 산출물과 테스트 케이스를 조회한다. 2. 테스트 시스템은 테스트 케이스를 순차적으로 실행한다. 3. 테스트 시스템은 각 ReqID 별 테스트 실행 결과(성공 또는 실패)와 에러 로그를 수집하여 시스템에 전달한다. 4. 시스템은 (App ID: [(ReqID: 결과, 로그), ...]) 형태로 DBMS 에 데이터 저장을 요청한다. 5. 시스템은 DBMS 로부터 저장 성공 메시지를 반환받으면, 사용자에게 테스트 수행이 완료됨을 반환한다. 예외흐름: 2-1. 실행 환경(컨테이너 등) 오류로 인해 테스트 수행 자체가 불가능한 경우: 1. 테스트 시스템은 환경 초기화를 수행한 후 최대 3 회 재실행을 시도한다. 2. 재실행을 3 회 이상 시도했음에도 실패할 경우 DBMS 에 시스템 오류 로그 저장을 요청하고 테스트 수행을 강제 중단한다.
후행 조건	테스트 실행 결과가 산출물 재생성 또는 최종 애플리케이션 제공 여부를 결정하기 위해 DBMS 에 저장된다.

2026 전기 졸업과제

● 클라우드 네이티브 애플리케이션 제공

유스케이스명	클라우드 네이티브 애플리케이션 제공
개요	모든 테스트를 통과한 최종 산출물을 패키징하여 사용자에게 파일 형태로 제공한다.
관련 액터	사용자, DBMS
선행 조건	테스팅 수행 결과의 모든 항목이 통과 상태여야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 DBMS로부터 특정 App ID의 소프트웨어 테스트 결과를 조회하여 테스트 결과가 100% 통과 상태임을 확인한다. 2. 시스템은 DBMS로부터 특정 App ID의 최종 산출물(소스 코드, 배포 파일)을 조회하여 ZIP 등의 압축 파일 객체로 패키징한다. 3. 시스템은 사용자 인터페이스에 파일 다운로드 경로 및 버튼을 생성하여 활성화한다. 4. 사용자가 다운로드 버튼을 클릭하여 결과물을 로컬 환경에 저장한다.
후행 조건	사용자는 최종 클라우드 네이티브 애플리케이션 파일을 확보하며 프로젝트가 종료된다.

3. 현실적 제약 사항 분석 결과 및 대안

1) 현실적 제약 사항

- ① 클라우드 네이티브 애플리케이션을 구성하는 다양한 클라우드 인프라의 구성 요소를 모두 고려하여 산출물을 테스트할 수 있는 데이터셋이 부족하다.
- ② 멀티 AI 에이전트 구성을 위해서 다양한 LLM 을 활용해야 하나, LLM 의 API 호출 비용 문제가 발생한다.
- ③ 클라우드 네이티브 애플리케이션에서 사용되는 다양한 프로그래밍 언어와 테스트 도구를 모두 지원하기 어렵다.

2) 대안

- ① 생성형 AI 를 활용하여 클라우드 네이티브 애플리케이션의 특성을 반영한 테스트 데이터로 대체한다.
- ② 경량 LLM 모델을 활용하여 API 호출 비용을 줄인다.
- ③ 프레임워크의 테스트 자동화 지원 범위를 특정 기술 스택(예: Java, Spring Boot)과 대중적인 단위 테스트 도구(예: JUnit)로 제한하여 핵심 기능 구현에 집중한다.

4. 시스템 구성

1) 시스템 구성도

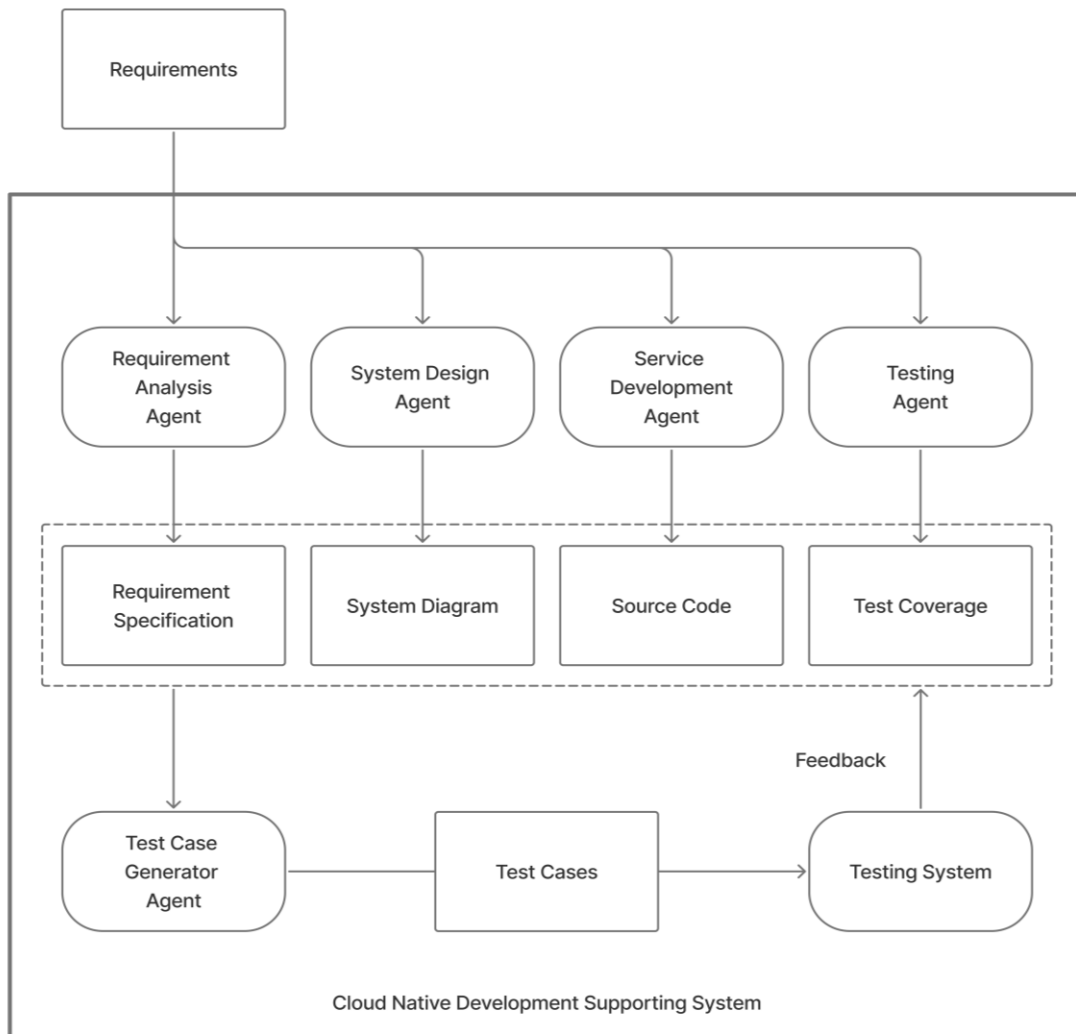


그림 2. 시스템 구성도

그림 2에서 본 시스템을 구성하는 기능으로는 클라우드 네이티브 애플리케이션 개발, 산출물 도출, 테스트 케이스 생성 및 검증, 테스트 수행, 테스트 결과 기반 산출물 재생성, 클라우드 네이티브 애플리케이션 제공이 있다.

시스템은 자연어로 입력된 클라우드 네이티브 애플리케이션에 대한 요구사항을 기반으로 클라우드 네이티브 애플리케이션을 개발한다. 먼저 시스템은 클라우드 네이티브 애플리케이션의 각 개발 단계별 산출물에 대한 요구사항을 입력받고, 각 요구사항을 만족할 수 있는 산출물을 생성하며, 각 산출물이 개별 요구사항과 부합하는지 확인하기 위해 요구사항 항목별 테스트 케이스를 생성한다. 시스템은 AI 에이전트가 생성한 테스트 케이스가 요구사항에 맞게 작성되었는지 사용자에게 승인을 받는다. 시스템은 생성된 테스트 케이스를 기반으로 테스트를 수행하고 테스트 성공 여부를 반환한다. 시스템은 테스트 결과에 따라 산출물을 재생성하거나 다음 개발 단계로 진행한다. 시스템은 최종적으로 개발된 클라우드 네이티브 애플리케이션의 산출물들을 사용자에게 파일 형태로 제공한다.

2) 개발 환경

그림 3 은 시스템을 개발할 환경의 구성을 나타낸다.

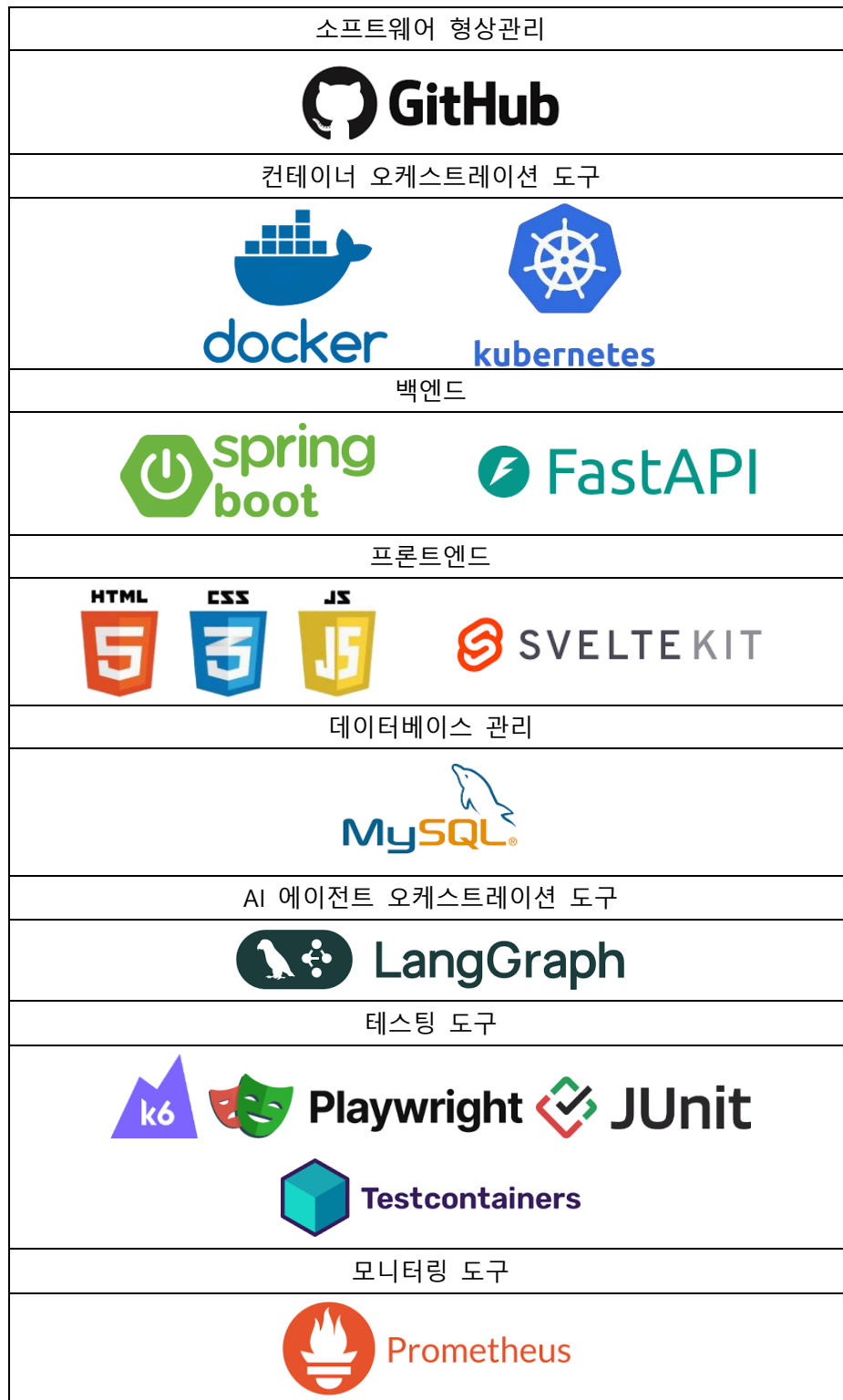


그림 3. 개발 환경 구성도

표 4 는 시스템 개발에 사용되는 관련 기술들을 나타낸다.

표 4. 개발 환경

개발환경	사용기술
소프트웨어 형상관리	GitHub
컨테이너 오케스트레이션 도구	Docker, Kubernetes
백엔드	Spring Boot, FastAPI
프론트엔드	HTML, CSS, JavaScript, SvelteKit
데이터베이스 관리	MySQL
AI 에이전트 오케스트레이션 도구	LangGraph
테스팅 도구	k6, Playwright, JUnit, Testcontainers
모니터링 도구	Prometheus

3) 사용 기술

① Docker

도커는 애플리케이션을 실행하는 데 필요한 코드, 라이브러리, 설정값 등을 하나의 실행 단위로 묶어 컨테이너 형태로 배포·운영할 수 있게 해주는 오픈소스 플랫폼이다. 이를 활용하면 개발자가 만든 실행 환경을 다른 시스템에서도 거의 동일하게 재현할 수 있어, 환경 차이로 인해 발생하는 문제를 줄일 수 있다. 또한 운영체제 커널을 공유하는 경량 가상화 방식을 사용하므로, 전통적인 가상머신보다 빠르고 효율적으로 서비스를 실행할 수 있다.

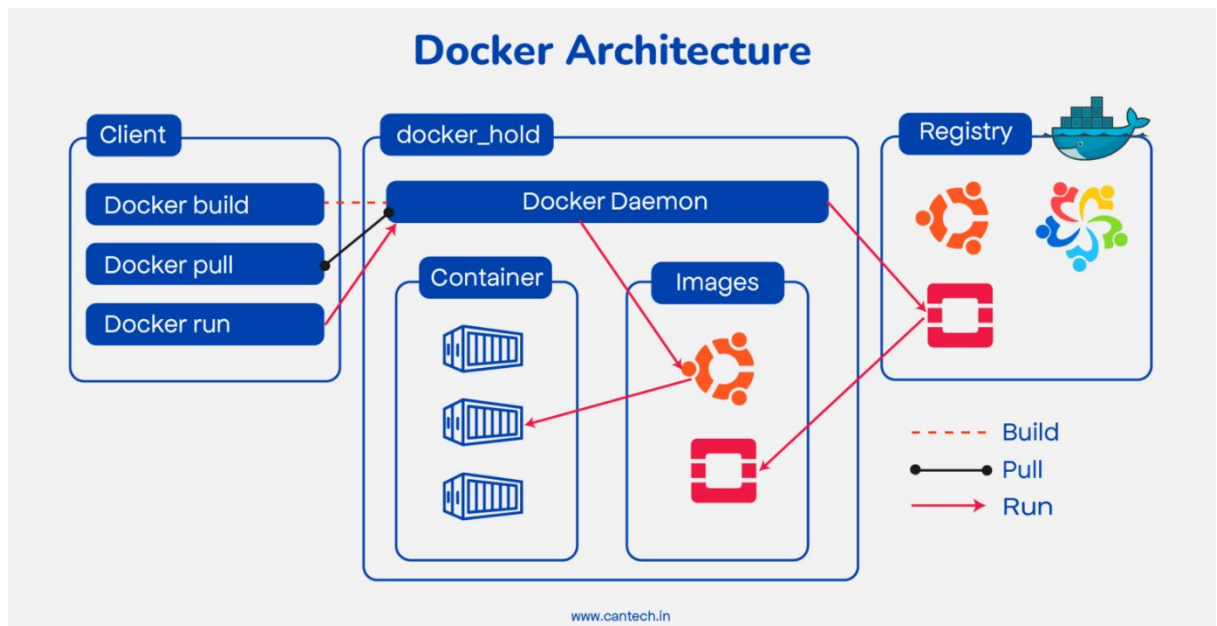


그림 4. Docker 아키텍처

도커의 주요 구성 요소는 다음과 같다.

- i) Engine: 도커 엔진은 도커 환경에서 실제 컨테이너 관련 작업을 처리하는 중심으로서, 시스템 내부에서 지속적으로 동작하면서 이미지 생성, 컨테이너 시작 및 종료, 네트워크 연결, 저장 공간 할당 등의 기능을 수행한다.
- ii) Client: 클라이언트는 사용자가 도커 기능을 활용할 수 있도록 해주는 명령 입력 도구이다. 일반적으로 터미널에서 입력하는 명령어를 통해 컨테이너 실행, 이미지 생성, 상태 확인 등의 작업을 요청할 수 있다.
- iii) Image: 이미지는 애플리케이션 실행에 필요한 요소들을 포함하고 있는 읽기 전용 템플릿이다. 하나의 이미지를 기반으로 여러 개의 컨테이너를 생성할 수 있으며, 실행 환경을 일관되게 유지하는 기반이 된다.
- iv) Container: 컨테이너는 이미지를 실행한 실제 동작 단위이다. 독립된 환경에서 프로그램이 실행되며, 다른 컨테이너와 분리된 상태를 유지하면서도 호스트 운영체제를 공유하기 때문에 호스트 자원을 효율적으로 활용할 수 있다.

v) Registry: 레지스트리는 이미지를 보관하고 공유하기 위한 저장 공간이다. 필요한 이미지를 가져올 수도 있고, 직접 생성한 이미지를 업로드하여 다른 환경에서 다시 사용할 수도 있다.

vi) Volume: 볼륨은 컨테이너 외부에 데이터를 저장할 수 있도록 지원하는 기능이다. 컨테이너가 삭제되거나 다시 생성되더라도 필요한 데이터를 유지할 수 있다는 점에서 중요하다.

vii) Network: 네트워크는 컨테이너 사이의 연결을 구성하는 기능이다. 이를 통해 서로 다른 컨테이너가 데이터를 주고받으며 하나의 서비스처럼 동작할 수 있다.

② Kubernetes

쿠버네티스는 컨테이너화된 애플리케이션의 배포, 운영, 확장, 관리를 자동화하기 위한 오픈소스 컨테이너 오케스트레이션 플랫폼이다. 다수의 컨테이너를 효율적으로 관리하고, 서비스의 안정성과 확장성을 높이기 위해 사용된다. 애플리케이션을 여러 서버 환경에 걸쳐 일관되게 배포하고 운영할 수 있도록 지원하며, 자동화된 운영 기능을 통해 복잡한 인프라 관리 부담을 줄이는 것을 목표로 한다.

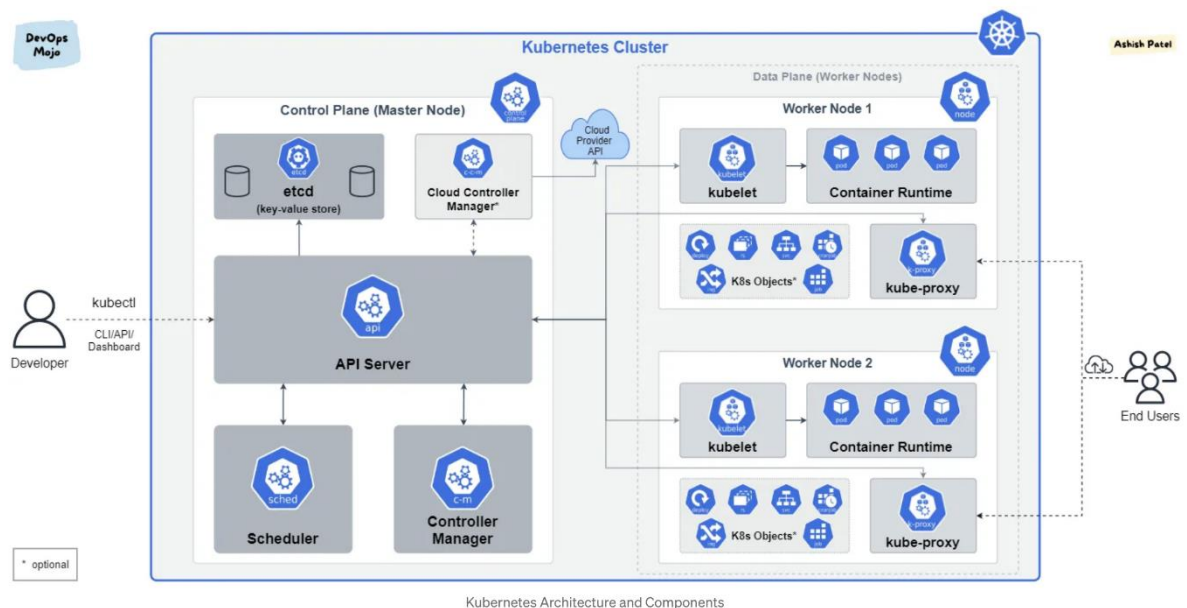


그림 5. Kubernetes 아키텍처

쿠버네티스의 주요 구성 요소는 다음과 같다.

i) Pod: 파드는 쿠버네티스에서 애플리케이션 컨테이너를 실제로 배포하고 실행하는 가장 기본적인 단위이다. 하나 이상의 컨테이너를 포함할 수 있으며, 같은 파드에 속한 컨테이너는 저장소와 네트워크를 공유할 수 있다.

ii) Node: 노드는 쿠버네티스 클러스터를 구성하는 기본 실행 단위인 서버 또는 가상머신을 의미한다. 각 노드는 컨테이너화된 애플리케이션을 실행하거나 클러스터의 전체 상태

를 관리하는 역할을 수행한다. 일반적으로 노드는 마스터 노드(Master Node)와 워커 노드(Worker Node)로 구분된다.

A. 마스터 노드는 쿠버네티스 클러스터의 제어와 관리 기능을 담당하는 노드로, 스케줄링, 클러스터 상태 관리, API 제공 등의 역할을 수행한다.

B. 워커 노드는 실제 애플리케이션 컨테이너가 배포되고 실행되는 노드로, 사용자의 서비스가 동작하는 실질적인 작업 공간 역할을 한다.

iii) Cluster: 클러스터는 여러 개의 노드를 하나로 묶어 관리하는 쿠버네티스의 실행 환경이다. 클러스터는 전체 애플리케이션과 자원을 통합적으로 관리하며, 여러 노드에 걸쳐 분산된 컨테이너를 안정적으로 운영할 수 있도록 한다.

iv) Deployment: 디플로이먼트는 애플리케이션의 배포 상태를 정의하고 관리하는 구성 요소이다. 사용자는 원하는 파드의 개수와 업데이트 방식 등을 설정할 수 있으며, 쿠버네티스는 이에 따라 파드를 생성하거나 갱신한다. 이를 통해 무중단 배포와 버전 업데이트를 체계적으로 수행할 수 있다.

v) Service: 서비스는 여러 파드에 대한 고정된 접근 경로를 제공하는 구성 요소이다. 파드는 생성과 삭제에 따라 주소가 달라질 수 있으나, 서비스는 일관된 네트워크 주소를 제공하여 외부 또는 내부에서 안정적으로 접근할 수 있게 한다. 또한 요청을 여러 파드에 분산하는 역할도 수행한다.

vi) ConfigMap 과 Secret: ConfigMap 은 애플리케이션 설정 정보를 저장하는 데 사용되며, Secret 은 비밀번호나 인증 정보와 같은 민감한 데이터를 저장하는 데 사용된다.

③ FastAPI

FastAPI 는 Python 을 기반으로 하는 오픈소스 웹 프레임워크로, API 를 빠르고 효율적으로 개발하기 위해 사용된다. 현대적인 Python 문법과 비동기 처리 방식을 지원하며, 높은 성능과 개발 편의성을 함께 제공하는 것을 목표로 한다. FastAPI 는 데이터 검증, 요청 처리, 문서 자동 생성 등의 기능을 통합적으로 지원하여 백엔드 서비스와 웹 API 개발에 널리 활용된다. FastAPI 의 주요 구성 요소는 다음과 같다.

i) Application: 애플리케이션은 FastAPI 프로젝트의 중심이 되는 객체로, 전체 API 서버를 구성하는 기본 단위이다. 이 객체를 통해 경로를 등록하고 요청 및 응답 처리를 수행할 수 있다.

ii) Path Operation: 경로 동작은 특정 URL 경로와 HTTP 메서드에 대해 실행할 동작을 정의하는 기능이다. 예를 들어 데이터 조회, 생성, 수정, 삭제와 같은 작업을 각각의 경로에 연결할 수 있다.

iii) Router: 라우터는 여러 API 경로를 기능별로 나누어 관리할 수 있도록 하는 구성 요소이다. 이를 활용하면 코드의 재사용성과 유지보수성을 높일 수 있으며, 특히 규모가 큰 애플리케이션에서 효율적인 경로 관리에 유용하다.

④ MySQL

MySQL 은 오픈소스 관계형 데이터베이스 관리 시스템으로, 데이터를 표 형태로 저장하고 관리하기 위해 사용된다. SQL 을 기반으로 데이터의 정의, 조작, 제어 기능을 수행하며, 다양한 웹 애플리케이션과 정보 시스템에서 널리 활용된다. MySQL 의 주요 구성 요소는 다음과 같다.

- i) Database: 데이터베이스는 여러 개의 테이블과 관련 객체를 포함하는 논리적인 저장 공간이다. 하나의 MySQL 서버 안에는 여러 데이터베이스를 생성할 수 있으며, 각 데이터베이스는 서로 다른 목적과 구조를 가진 데이터를 독립적으로 관리할 수 있다.
- ii) Table: 테이블은 MySQL 에서 데이터를 저장하는 기본 단위이다. 데이터는 행과 열의 구조로 저장되며, 각 열은 특정 속성을 나타내고 각 행은 하나의 실제 데이터 항목을 의미한다. 테이블은 데이터의 구조를 명확하게 정의하고 체계적으로 관리하는 데 중요한 역할을 한다.
- iii) Column: 열은 테이블을 구성하는 속성 단위로, 데이터의 종류와 형식을 정의한다. 예를 들어 이름, 번호, 날짜와 같은 정보를 각각의 열로 구성할 수 있다. 각 열은 데이터 타입을 가지며, 이를 통해 저장할 수 있는 값의 형태를 제한할 수 있다.
- iv) Row: 행은 테이블에 저장되는 개별 데이터 레코드를 의미한다. 하나의 행은 여러 열의 값을 포함하며, 하나의 객체 또는 하나의 정보 단위를 나타낸다. 이를 통해 실제 데이터를 구조화된 형태로 저장하고 관리할 수 있다.
- v) Key: 키는 테이블 내의 데이터를 식별하거나 테이블 간의 관계를 설정하기 위한 요소이다. 기본 키는 각 행을 고유하게 구분하는 역할을 하며, 외래 키는 다른 테이블의 기본 키를 참조하여 테이블 간 관계를 형성한다. 이러한 키 구조를 통해 데이터 검색의 효율성을 높이고 데이터 무결성을 유지할 수 있다.
- vi) Query: 쿼리는 데이터베이스에 특정 작업을 요청하는 명령문이다. 사용자는 쿼리를 통해 데이터를 조회하거나 삽입, 수정, 삭제할 수 있으며, 필요한 조건을 설정하여 원하는 결과를 얻을 수 있다. 쿼리는 MySQL 을 실제로 활용하는 데 있어 핵심적인 수단이다.

⑤ SvelteKit

SvelteKit 은 Svelte 를 기반으로 하는 오픈소스 웹 애플리케이션 프레임워크로, 빠르고 효율적인 사용자 인터페이스와 서버 기능을 함께 구현할 수 있도록 지원한다. SvelteKit 은 정적 사이트 생성, 서버 사이드 렌더링, 클라이언트 사이드 렌더링을 모두 지원하며, 현대적인 웹 애플리케이션 개발에 필요한 라우팅, 데이터 로딩, 페이지 렌더링 기능을 통합적으로 제공하는 것을 목표로 한다. SvelteKit 의 주요 구성 요소는 다음과 같다.

- i) Component: 컴포넌트는 SvelteKit 애플리케이션의 기본적인 구성 요소로, 화면을 이루는 각 부분을 독립적으로 정의하는 단위이다. 각 컴포넌트는 마크업, 스타일, 로직을 하나의 파일 안에서 함께 작성할 수 있으며, 필요에 따라 다른 컴포넌트와 조합하여 복잡한 사용자 인터페이스를 구성할 수 있다.

ii) Route: 라우트는 사용자가 접근하는 페이지 경로를 의미하며, SvelteKit에서는 파일과 폴더 구조를 기반으로 구성된다. 각 라우트는 하나의 페이지 또는 기능 단위를 나타내며, 정적인 경로와 동적인 경로를 모두 설정할 수 있다.

iii) Load Function: 로드 함수는 페이지가 렌더링되기 전에 필요한 데이터를 불러오는 기능을 담당한다. 이를 활용하면 서버 또는 외부 데이터 소스로부터 필요한 정보를 미리 가져와 페이지에 전달할 수 있다. 이 기능은 페이지 초기 로딩 시 필요한 데이터를 효율적으로 처리할 수 있게 한다.

iv) Layout: 레이아웃은 여러 페이지에서 공통으로 사용되는 구조를 정의하는 요소이다. 예를 들어 내비게이션 바, 헤더, 푸터와 같은 공통 UI를 레이아웃으로 구성할 수 있다. 이를 통해 반복되는 코드를 줄이고, 여러 페이지에서 일관된 사용자 인터페이스를 유지할 수 있다.

v) Endpoint: 엔드포인트는 서버 측에서 요청을 처리하기 위한 기능 요소이다. 이를 통해 데이터 조회, 저장, 수정, 삭제와 같은 작업을 서버에서 수행할 수 있다. SvelteKit은 이러한 서버 기능을 프레임워크 내부에서 함께 제공한다.

⑥ LangGraph

LangGraph는 대규모 언어 모델 기반 애플리케이션의 실행 흐름을 그래프 구조로 설계하고 관리할 수 있도록 지원하는 프레임워크이다. 복잡한 에이전트 시스템, 다단계 추론 과정, 상태 기반 워크플로우를 보다 명확하고 안정적으로 구현하기 위해 사용된다. LangGraph는 단순한 순차 실행을 넘어, 조건 분기, 반복 처리, 상태 유지, 여러 작업 노드 간 연결을 체계적으로 구성하는 것을 목표로 한다.

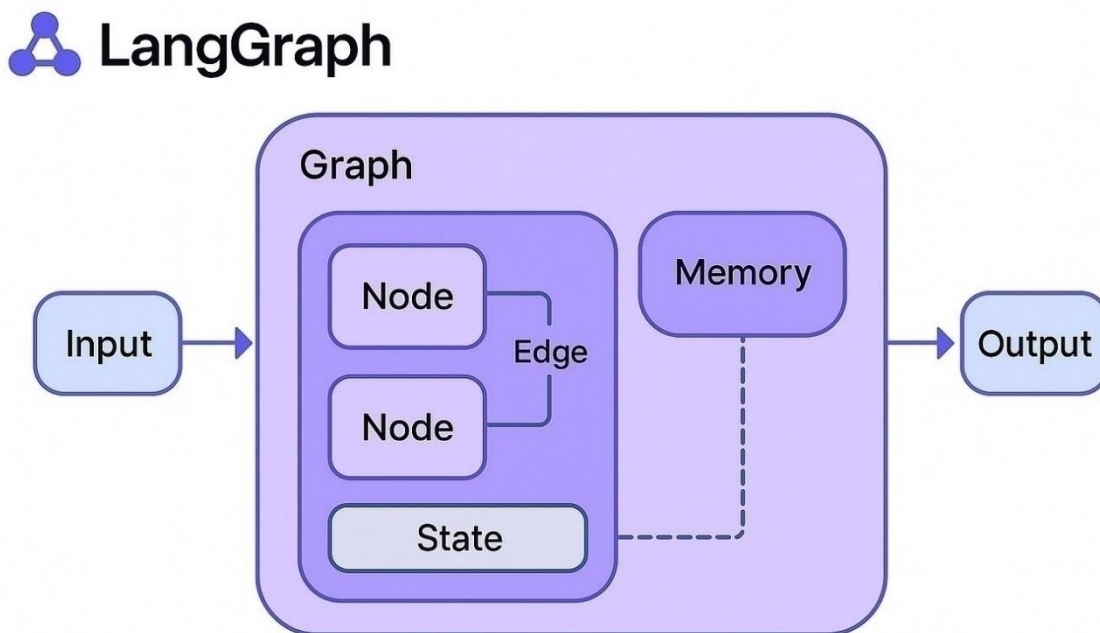


그림 6. LangGraph 아키텍처

LangGraph의 주요 구성 요소는 다음과 같다.

- i) Node: 노드는 그래프에서 개별 작업을 수행하는 단위이다. 하나의 노드는 언어 모델 호출, 데이터 처리, 외부 도구 실행, 조건 판단 등 특정 기능을 담당할 수 있다. 애플리케이션의 실제 실행 로직은 각각의 노드 안에 정의된다.
- ii) Edge: 엣지는 노드와 노드 사이의 연결 관계를 의미한다. 각 작업이 어떤 순서와 조건으로 이어지는지를 정의하며, 실행 흐름을 결정하는 중요한 역할을 한다. 이를 통해 단순 직렬 처리뿐 아니라 다양한 경로 구성이 가능하다.
- iii) State: 상태는 그래프 전체에서 공유되거나 갱신되는 데이터 구조를 의미한다. 각 노드는 상태를 읽고 수정할 수 있으며, 이를 통해 이전 단계의 결과를 다음 단계로 전달할 수 있다. 상태는 LangGraph의 흐름 제어와 문맥 유지에서 핵심적인 역할을 한다.
- iv) Memory: 메모리는 그래프 실행 과정에서 대화 이력, 이전 작업 결과, 사용자 설정 등 장기적 또는 보조적으로 유지되어야 하는 정보를 저장하고 활용하는 기능이다. 이를 통해 단순한 현재 상태 관리뿐 아니라, 여러 실행 단계에 걸친 문맥 유지와 개인화된 응답, 반복 작업의 일관성 확보가 가능하다.
- v) Conditional Routing: 조건부 라우팅은 특정 조건에 따라 다음 실행 노드를 다르게 선택하는 기능이다. 작업 결과나 상태값을 기반으로 분기 처리를 구현할 수 있으며, 복잡한 의사결정 흐름을 구성하는 데 활용된다.
- vi) Loop: 루프는 특정 노드나 경로를 반복적으로 실행하는 구조를 의미한다. 목표 결과나 나올 때까지 작업을 재시도하거나, 단계적으로 응답 품질을 향상시키는 방식의 처리에 적합하다. 이를 통해 반복 추론 기반 애플리케이션을 구현할 수 있다.

5. 개발 일정 및 역할 분담

1) 개발 일정

표 5 는 시스템 개발 일정을 나타낸다.

표 5. 개발 일정

수행 내용 \ 기간	5 월		6 월				7 월				8 월				9 월		
	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3
필요 지식 습득	All																
소프트웨어 개발 과정별 요구사항 수립 체계 구축			황수환														
AI 에이전트 프롬 프트 및 데이터 전달 포맷 설계			김재우														
소프트웨어 특성 및 개발 과정 산 출물 저장을 위한 DBMS 설계			이상현														
소프트웨어 개발 단계별 산출물에 대한 테스트 케이 스 도출 기능 개 발					황수환												
요구사항-테스트 추적 관리 기능 개발							김재우										
AI 에이전트 기반 소프트웨어 개발 파이프라인 구축							이상현, 황수환										
단계별 테스트 자 동화 및 테스트 결과 기반 소프트 웨어 개선 기능 개발							이상현										
소프트웨어 테스 팅 결과 모니터링 시스템 구축										김재우							
시스템 API 개발											이상현, 황수환						
시스템 UI 개발											김재우						
테스트 및 보완													All				
최종보고서 작성															All		

2) 역할 분담

표 6 은 시스템 개발에서 개인별 역할을 나타낸다.

표 6. 역할 분담

이름	역할
공통	필요 지식 습득, 테스트 및 보완, 최종보고서 작성
김재우	시스템 UI 개발, 요구사항·테스트 추적 관리 기능 개발, 소프트웨어 테스트 결과 모니터링 시스템 구축, AI 에이전트 프롬프트 및 데이터 전달 포맷 설계
이상현	시스템 API 개발, AI 에이전트 기반 소프트웨어 개발 파이프라인 구축, 단계별 테스트 자동화 및 테스트 결과 기반 소프트웨어 개선 기능 개발, 소프트웨어 특성 및 개발 과정 산출물 저장을 위한 DBMS 설계
황수환	시스템 API 개발, AI 에이전트 기반 소프트웨어 개발 파이프라인 구축, 소프트웨어 개발 단계별 산출물에 대한 테스트 케이스 도출 기능 개발, 소프트웨어 개발 과정별 요구사항 수립 체계 구축